



PROJECT PRESENTATION

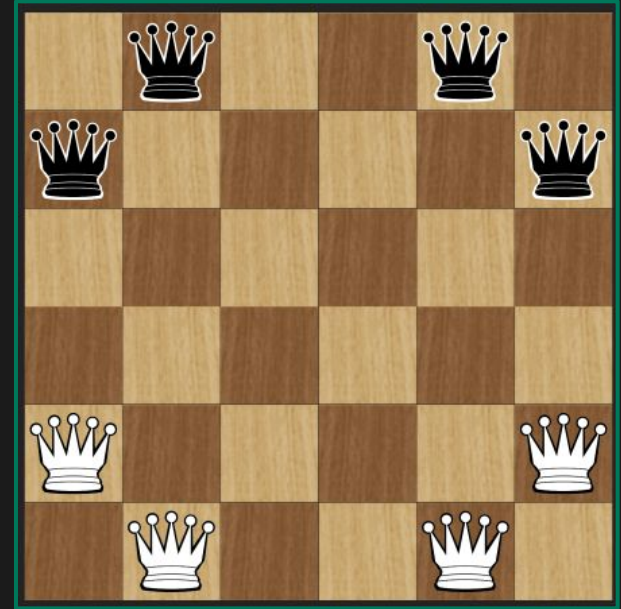
GAME OF THE AMAZONS

Available on:
<https://github.com/ehadziabdic/GameOfTheAmazons>



▪ About the Game of Amazons

- Invented in 1988 by Walter Zamkaskas of Argentina
- Strategy game for two players, combining the movement of chess with the added space-restriction of Go.
- Each player controls four Amazons (Queens).
- Battle for space where players try to isolate their opponent by blocking of sections of board with arrows.
- The Objective - be the last player capable of making a move.
- This makes the Game of Amazons a fight for survival.



01

GAME RULES

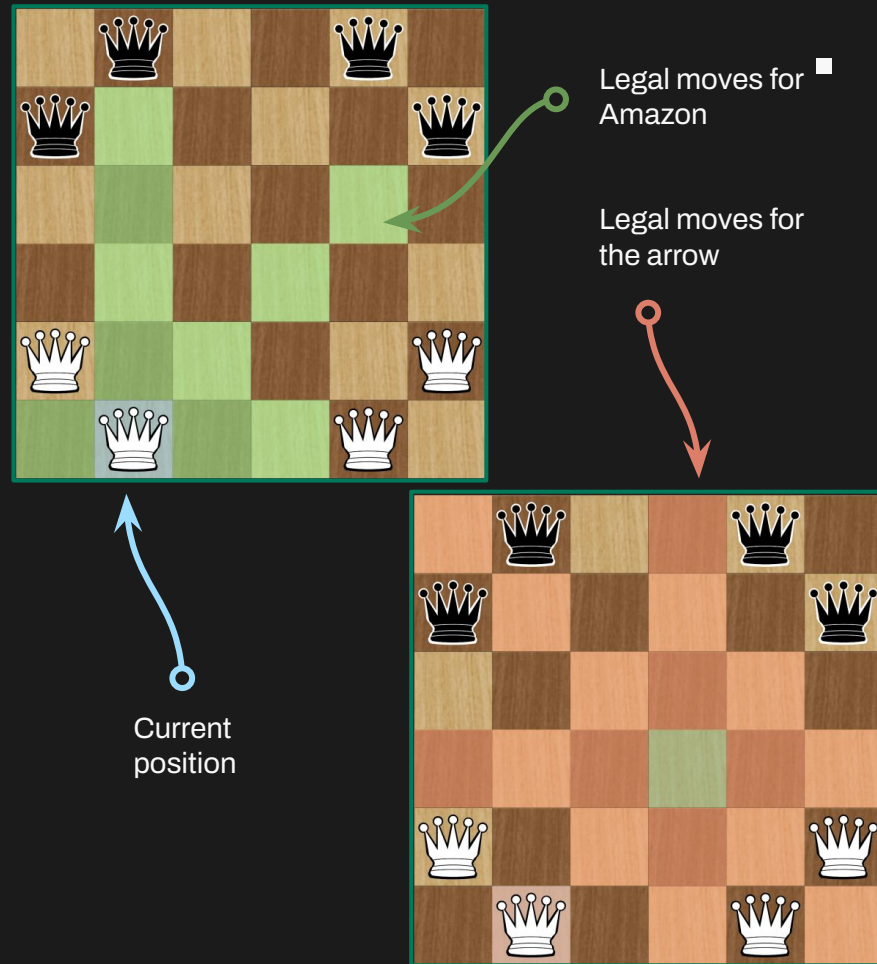


■ Dev: Muhammed Pašić



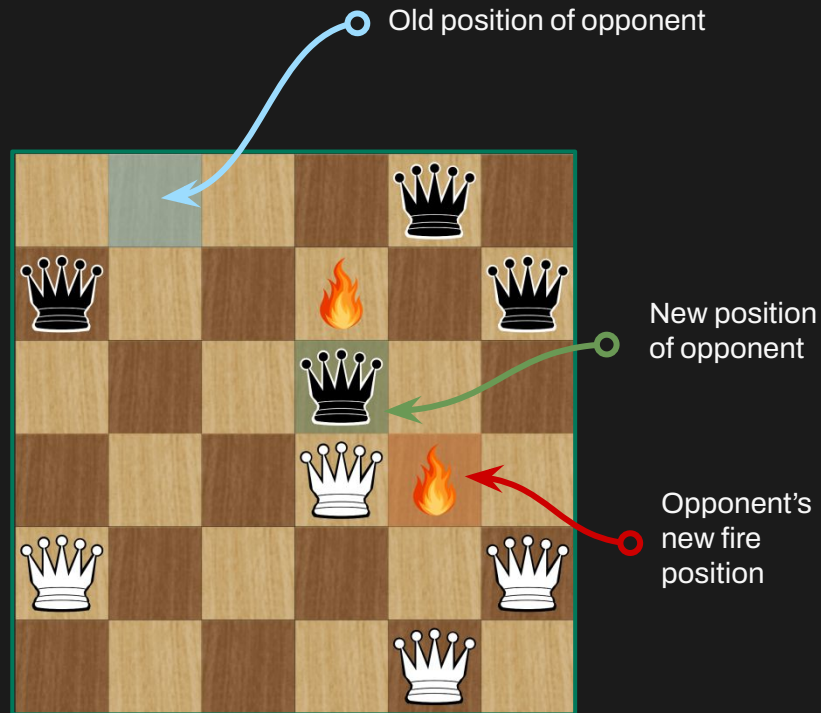
Game Rules - Player's Turn

- White goes first.
- Shift one Amazon any distance (vertically, horizontally or diagonally) like a chess queen.
- From this new position, fire your arrow to any empty square using the same queen like movement.



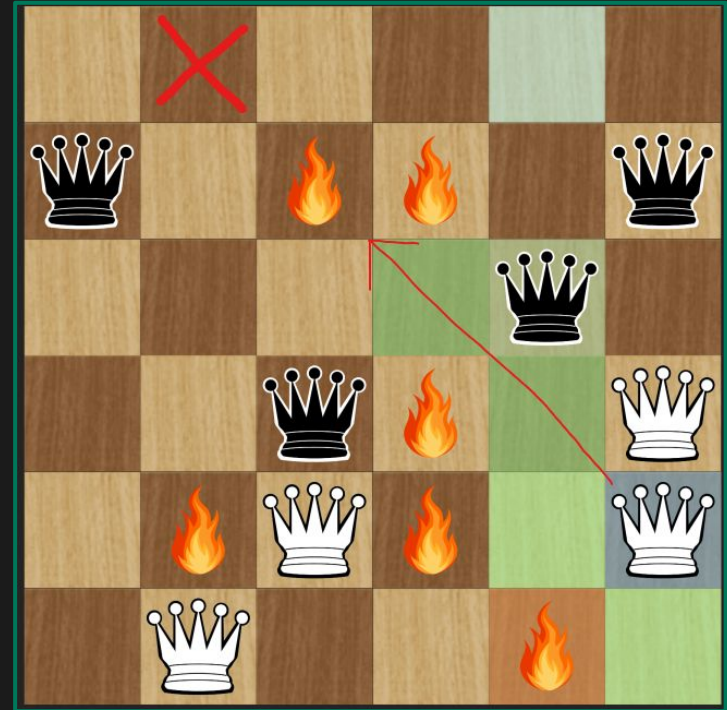
Game Rules - Player's Turn

- White goes first.
- Shift one Amazon any distance (vertically, horizontally or diagonally) like a chess queen.
- From this new position, fire your arrow to any empty square using the same queen like movement.



Game Rules - Restrictions

- Once an arrow lands on a square, that square is off limits for the remainder of the game.
- Neither Amazons nor arrows can pass through or jump over blocked or occupied squares.



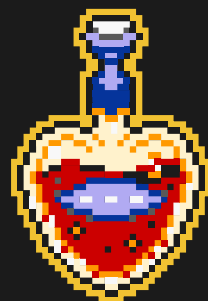
Game Rules – Victory

- The game goes on until a player is walled in and has no legal moves left.



02

GAME INTERFACE

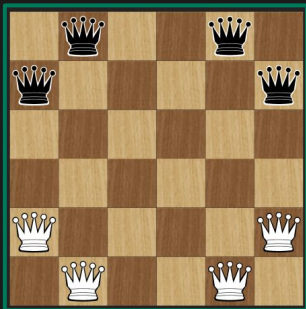


■ Dev: Emin Hadžiabdić



Game interface - Board Grid

- ✓ Nested loop iterates every board position
- ✓ **Checkerboard pattern:** `(row + col) % 2`
- ✓ **Image themes:** Draw pre-loaded textures
- ✓ **Color themes:** Use `Shape::drawRect()`
- ✓ **Efficient:** Single draw call per tile



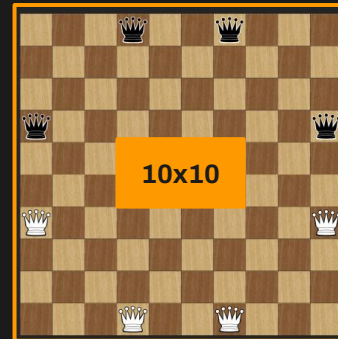
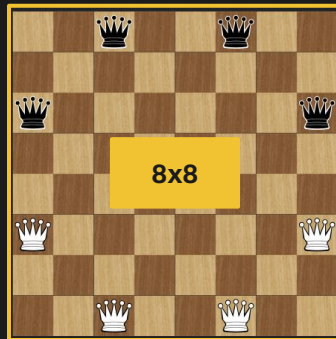
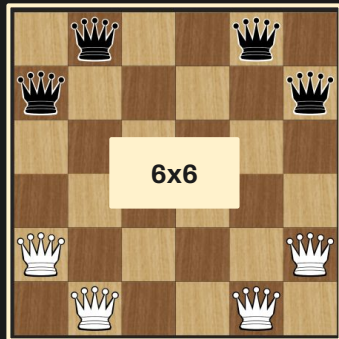
```
1 void drawBoardGrid() const {
2     const int gridSize = _state->boardSize();
3     const bool hasImageTheme = (_boardStyle == BoardStyle::Wooden ||
4                                 _boardStyle == BoardStyle::Ice ||
5                                 _boardStyle == BoardStyle::Stone ||
6                                 _boardStyle == BoardStyle::Diamond ||
7                                 _boardStyle == BoardStyle::Tournament);
8
9     for (int row = 0; row < gridSize; ++row) {
10        for (int col = 0; col < gridSize; ++col) {
11            Position pos{row, col};
12            auto cell = cellRect(pos);
13            bool isLightSquare = (row + col) % 2 == 0;
14
15            if (hasImageTheme) {
16                if (isLightSquare) {
17                    _lightTileImage.draw(cell);
18                } else {
19                    _darkTileImage.draw(cell);
20                }
21            } else {
22                // Color-based rendering
23                td::ColorID color = getColorForSquare(isLightSquare);
24                gui::shape::drawRect(cell, color);
25            }
26        }
27    }
28 }
```



Game interface - Board Sizes

We implemented **three distinct board sizes** to cater to different player skill levels and time constraints.

- **6x6 (Small)**: Designed for fast-paced "Blitz" games
- **8x8 (Medium)**: A balanced, **standard competitive** size with moderate complexity.
- **10x10 (Large)**: Designed for "**Professional**" games, offering maximum strategic depth.



Game interface - Board Style

Available Themes:

- Wooden
 - B&W
 - Tournament
 - Ice
 - Diamond
 - Stone
 - Bubblegum
 - Custom
- } Provide **professional** and **traditional** look
- } Provide more **modern** feel
- Exclusive theme just for **female** players
- “We didn't just give users colors; we gave them a **tool to create their own**.”



Inspired by [Chess.com](https://www.chess.com)



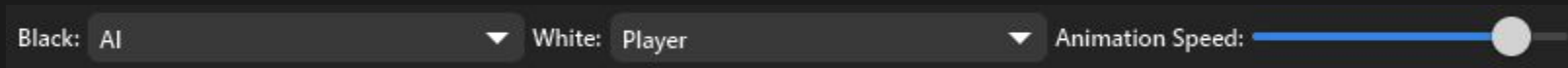
▪ Game interface – Game Modes

Matchmaking Modes:

- Player vs Player → Local multiplayer for social play.
- Player vs AI → Test skills against our Minimax engine.
- AI vs AI → "Spectator Mode" to observe the AI's decision-making

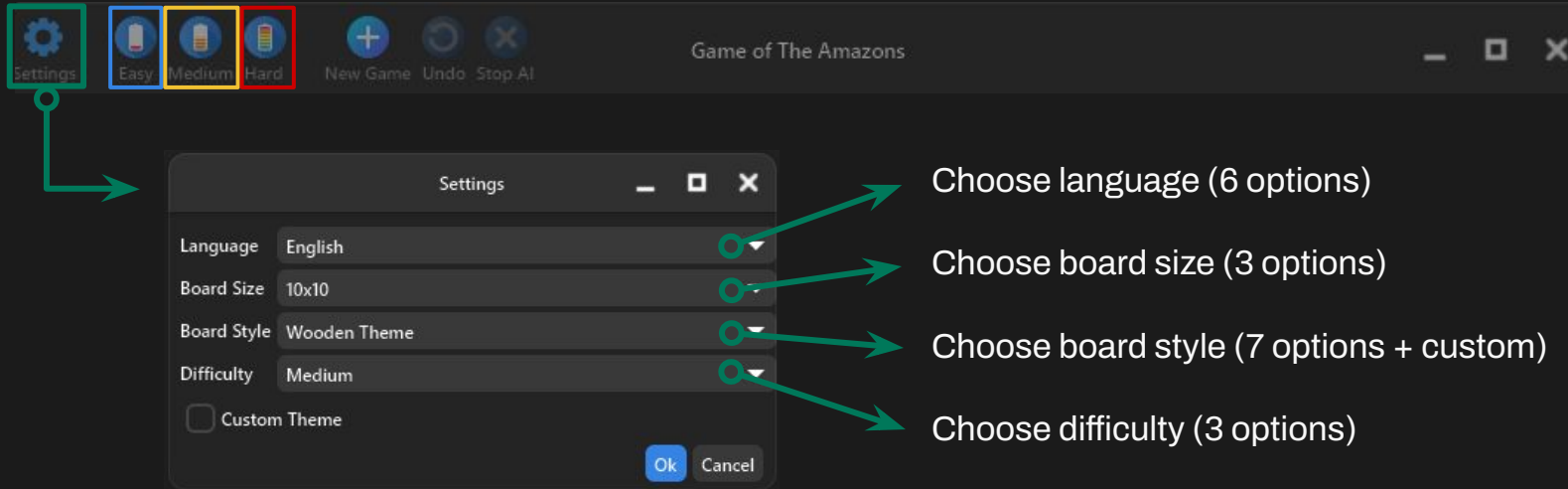
Animation Speed Slider:

A major UX addition that allows users to adjust the **pace for AI vs AI** matches from slow, cinematic movement to **near-instantaneous play**.



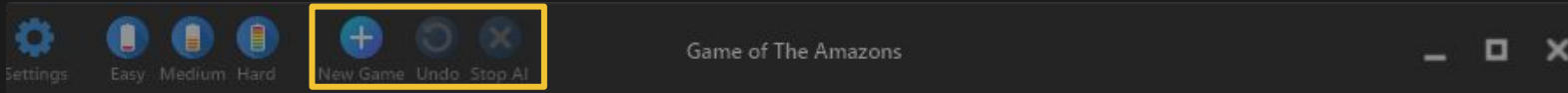
Game interface - Game Controls

- **Easy:** The AI configured to be **Easy Beatable**, for beginners.
- **Medium:** The AI configured to be **Hardly Beatable**, for intermediate players.
- **Hard:** The AI configured to be **Almost Unbeatable**, for an expert-level opponent.



▪ Game interface – Game Controls

- **New Game Button:** Resets the board state
- **Undo Button:** Allows players to **roll back queen** movement if they **don't want to make arrow placement**.
- **Stop AI Button:** Provides a **"safe exit"** by interrupting the **Minimax** search threads.



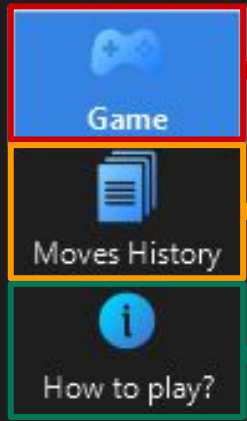
- **Status bar:** Acts as a real-time guide for the player

STATUS: Select a queen to move



• Game interface - Navigator

We implemented a three-page system to keep the interface clean and organized while providing all necessary information to the player.



Game Page

- The primary interactive layer where the board is rendered, animations occur (explosions, slides), and the actual gameplay takes place.

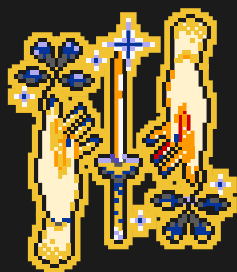
Move History Page

- A log that records every coordinate-to-coordinate move and arrow shot. This allows players to review their strategy or see exactly where the AI outmaneuvered them.

Rules & Info Page

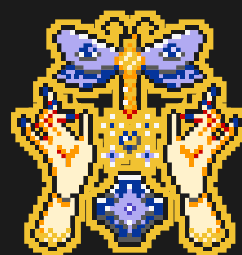
- An in-game manual that ensures that even players unfamiliar with the Game of the Amazons can start playing immediately without external help.





03

GAME ENGINE



Game Engine - Core Algorithm

→ Minimax Algorithm

- ◆ What: The core decision-making logic that predicts future game states.
- ◆ How: It recursively simulates moves. It assumes the AI wants to maximize the score and the opponent wants to minimize it.

→ Alpha-Beta Pruning

- ◆ What: An optimization to make the search faster.
- ◆ How: If the AI finds a move that is clearly worse than a previous option, it immediately stops searching that branch (the `if (alpha >= beta) break;` line).



```
1  inline int minimax(GameState& state, int depth, int alpha, int beta, Player maximizingPlayer,
2  Player perspective, std::size_t moveCap, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {
3
4  if (cancel && cancel->load()) throw SearchCanceled();
5
6  bool terminal = depth == 0 || detail::isTerminal(state);
7  if (terminal) {
8      GameState evalState = state;
9      evaluateWinState(evalState);
10     return detail::evaluateTerminal(evalState, perspective);
11 }
12
13 Player current = state.currentPlayer();
14 bool isMaximizing = current == maximizingPlayer;
15 auto moves = generateMovesForPlayer(state, current, moveCap);
16
17 if (moves.empty()) {
18     GameState evalState = state;
19     evaluateWinState(evalState);
20     return detail::evaluateTerminal(evalState, perspective);
21 }
22
23 if (isMaximizing) {
24     int value = std::numeric_limits<int>::min();
25     for (const auto& move : moves) {
26         if (cancel && cancel->load()) throw SearchCanceled();
27         GameState next = state.clone();
28         applyMove(next, move);
29         int child = minimax(next, depth - 1, alpha, beta, maximizingPlayer, perspective, moveCap, difficulty, cancel);
30         value = std::max(value, child);
31         alpha = std::max(alpha, value);
32         if (alpha >= beta) break;
33     }
34     return value;
35 }
36
37 int value = std::numeric_limits<int>::max();
38 for (const auto& move : moves) {
39     if (cancel && cancel->load()) throw SearchCanceled();
40     GameState next = state.clone();
41     applyMove(next, move);
42     int child = minimax(next, depth - 1, alpha, beta, maximizingPlayer, perspective, moveCap, difficulty, cancel);
43     value = std::min(value, child);
44     beta = std::min(beta, value);
45     if (alpha >= beta) break;
46 }
47 return value;
48 }
```

Game Engine - Search Constraints & Difficulty Scaling.

→ Search Depth

- ◆ What: Determines how many moves into the future the AI looks.
- ◆ How: * Easy: 2 moves ahead. Fast, but misses long-term traps.
- ◆ Medium/Hard: 3 moves ahead. This is the "sweet spot" for high-level play without long wait times.

→ Move Capping

- ◆ What: Limits the "Branching Factor" (the number of moves explored per turn).
- ◆ How: * In Amazons, one turn can have hundreds of possibilities. We cap this at 100–150 moves depending on difficulty.
 - Why: This prevents the AI from "over-thinking" irrelevant moves, keeping the performance cost low and the game responsive even on weaker hardware.

```
1 inline int depthForDifficulty(Difficulty difficulty) {  
2     switch (difficulty) {  
3         case Difficulty::Easy: return 2;  
4         case Difficulty::Medium: return 3;  
5         case Difficulty::Hard:  
6             default: return 3;  
7     }  
8 }  
9  
10 inline std::size_t moveCapForDifficulty(Difficulty difficulty) {  
11     switch (difficulty) {  
12         case Difficulty::Easy: return 100;  
13         case Difficulty::Medium: return 120;  
14         case Difficulty::Hard:  
15             default: return 150;  
16     }  
17 }
```

Game Engine - Heuristic Evaluation

→ Territory Score (Hard Mode)

- ◆ What: Calculates which player "owns" more of the board.
- ◆ How: Uses a Flood Fill algorithm (BFS) to count how many tiles our Queens can reach before the opponent.

```
1 inline int territoryScore(const GameState& state, Player player) {  
2     Player opponent = getOpponent(player);  
3     std::size_t playerReachable = 0;  
4     std::size_t opponentReachable = 0;  
5  
6     for (const auto& pos : scanForQueens(state, player)) {  
7         playerReachable += floodFillReachableTiles(state, pos);  
8     }  
9  
10    for (const auto& pos : scanForQueens(state, opponent)) {  
11        opponentReachable += floodFillReachableTiles(state, pos);  
12    }  
13  
14    return static_cast<int>(playerReachable) - static_cast<int>(opponentReachable);  
15 }
```

→ Spatial Influence (Medium & Hard)

- ◆ What: Encourages the AI to control the center of the board.
- ◆ How: Calculates distance from the center. Center squares give higher scores; corners give lower scores.

```
1 inline int spatialInfluenceScore(const GameState& state, Player player) {  
2     const auto& board = state.board();  
3     int dim = board.dimension();  
4  
5     if (dim == 0) return 0;  
6  
7     double center = (dim - 1) / 2.0;  
8  
9     auto positionalValue = [&](const Position& pos) {  
10        // Distance from center  
11        double dist = std::abs(pos.row - center) + std::abs(pos.col - center);  
12        double maxDist = 2.0 * (dim - 1);  
13        // Normalized: 1.0 = Center, 0.0 = Corner  
14        double normalized = 1.0 - (dist / maxDist);  
15  
16        // Factor in mobility slightly to ensure the spot isn't a trap  
17        auto mobility = gatherReachableTiles(board, pos).size();  
18        return static_cast<double>(mobility) * 0.25 + normalized * 10.0;  
19    };
```



Game Engine - Performance & Difficulty Handling

→ Move Ordering

- ◆ What: Speeds up the AI by guessing which moves are best before checking them fully.
- ◆ How: We calculate a quick heuristic score for all moves and sort them. Checking good moves first makes Alpha-Beta pruning much more effective.

→ Selective Deepening (Hard Mode)

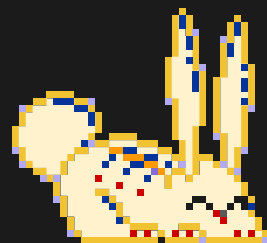
- ◆ What: Allows "Hard" difficulty to be smart but still responsive (fast).
- ◆ How:
 - The Top 6 moves are searched to Full Depth (Deep).
 - The remaining moves are searched to Shallow Depth (Fast).
 - This avoids wasting time on obviously bad moves.



```
1 inline Move getBestMove(const GameState& state, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {
2     ... rest of implementation
3
4     // Move Ordering
5     for (const auto& move : moves) {
6         GameState next = rootState.clone();
7         applyMove(next, move);
8         // Use the difficulty-specific evaluation
9         int heuristic = evaluate(next, perspective, difficulty);
10        scored.push_back({ move, heuristic });
11    }
12
13    std::sort(scored.begin(), scored.end(), [](const ScoredMove& a, const ScoredMove& b) {
14        return a.heuristic > b.heuristic;
15    });
16
17    Move bestMove = scored.front().move;
18    int bestScore = std::numeric_limits<int>::min();
19
20    int primaryDepth = std::max(0, searchDepth - 1);
21    int shallowDepth = std::max(0, primaryDepth - 1);
22
23    // Variable Depth Logic for Hard: Only search the top 6 moves deeply
24    std::size_t deepSlots = (difficulty == Difficulty::Hard && scored.size() > 6) ? 6 : scored.size();
25
26    for (std::size_t idx = 0; idx < scored.size(); ++idx) {
27        const auto& entry = scored[idx];
28        GameState next = rootState.clone();
29        applyMove(next, entry.move);
30
31        int depthForMove = primaryDepth;
32        if (difficulty == Difficulty::Hard && idx >= deepSlots) {
33            depthForMove = shallowDepth;
34        }
35
36        int score = minimax(next, depthForMove, std::numeric_limits<int>::min(),
37                           std::numeric_limits<int>::max(), maximizingPlayer, perspective, moveCap, difficulty, cancel);
38
39        if (score > bestScore) {
40            bestScore = score;
41            bestMove = entry.move;
42        }
43    }
44
45    return bestMove;
46 }
```

04

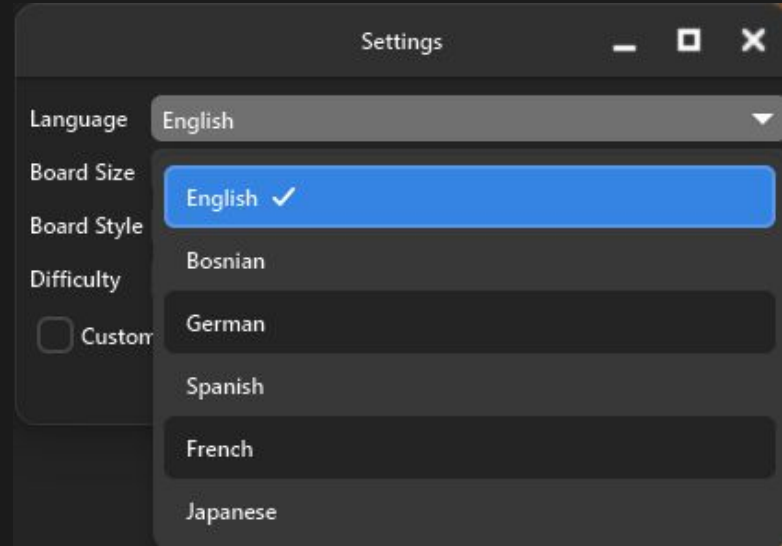
FEATURES



Available Languages

- The game is designed to be inclusive and it offers full support for diverse global communities.
- Multilingual support expands the potential user base significantly.
- Our application currently features professional localization for six major languages:

- English
- Bosnian
- German
- Spanish
- French
- Japanese (Kanji)



■ Original Features vs Additional Features

- ✓ **Core Game Rules:** Queen and Arrow placement, Path logic.
- ✓ **AI Engine:** Minimax with Alpha-Beta pruning.
- ✓ **Heuristics:** Spatial Influence & Territory Score
- ✓ **Difficulty Levels:** Easy, Medium, and Hard.
- ✓ **Board Sizes:** Small (6x6), Medium (8x8), Large (10x10)
- ✓ **Basic GUI:** Responsive interface, interactive Queens, move highlights.
- ✓ **Primary Themes:** Wooden and Black & White board styles.
- ✓ **Audio/Visuals:** Core animations for movement/explosions and basic sound effects.
- ✓ **Dual Language Support:** Interface available in Bosnian and English.
- ✓ **Expanded Theme Library:** Six additional visual styles.
- ✓ **Custom Theme Creator:** Integrated color pickers allowing users to define their own tile colors.
- ✓ **Additional Modes:** PvP and Spectator mode
- ✓ **Multi-Language Expansion:** Added support for total 6 languages.
- ✓ **Undo System:** A state-stack implementation that allows players to take back their last move.
- ✓ **Stop AI Functionality:** Safety feature to stop deep calculations safely.
- ✓ **Animation Speed Control:** Control of the pace of visual effects.
- ✓ **Move History Log:** A text-based record of all actions during the match.
- ✓ **Page System:** A three-page interface layout for Game, History, and Rules.
- ✓ **Full Compatibility:** App is compatible with windows, mac-os and linux





THANK YOU FOR YOUR ATTENTION



Download for Windows



Download for Linux



Download for MacOS Intel



Download for MacOS Silicon

